AI/ML India Market-Research Agent — Architecture Plan

Status: 🟡 DRAFT v0.1 (for review — not final) **Date:** 2026-06-16 **Owner:** IK India Marketing **Goal:** Autonomously reproduce the "CK's AI/ML Market Research India" 5-tab workbook (Social Trends, Blog & Publications, Job Market Demand, Competitor Intelligence, Executive Summary) on a recurring schedule, replacing the manual 2024–2025 research effort.

---

1. Objective & Scope

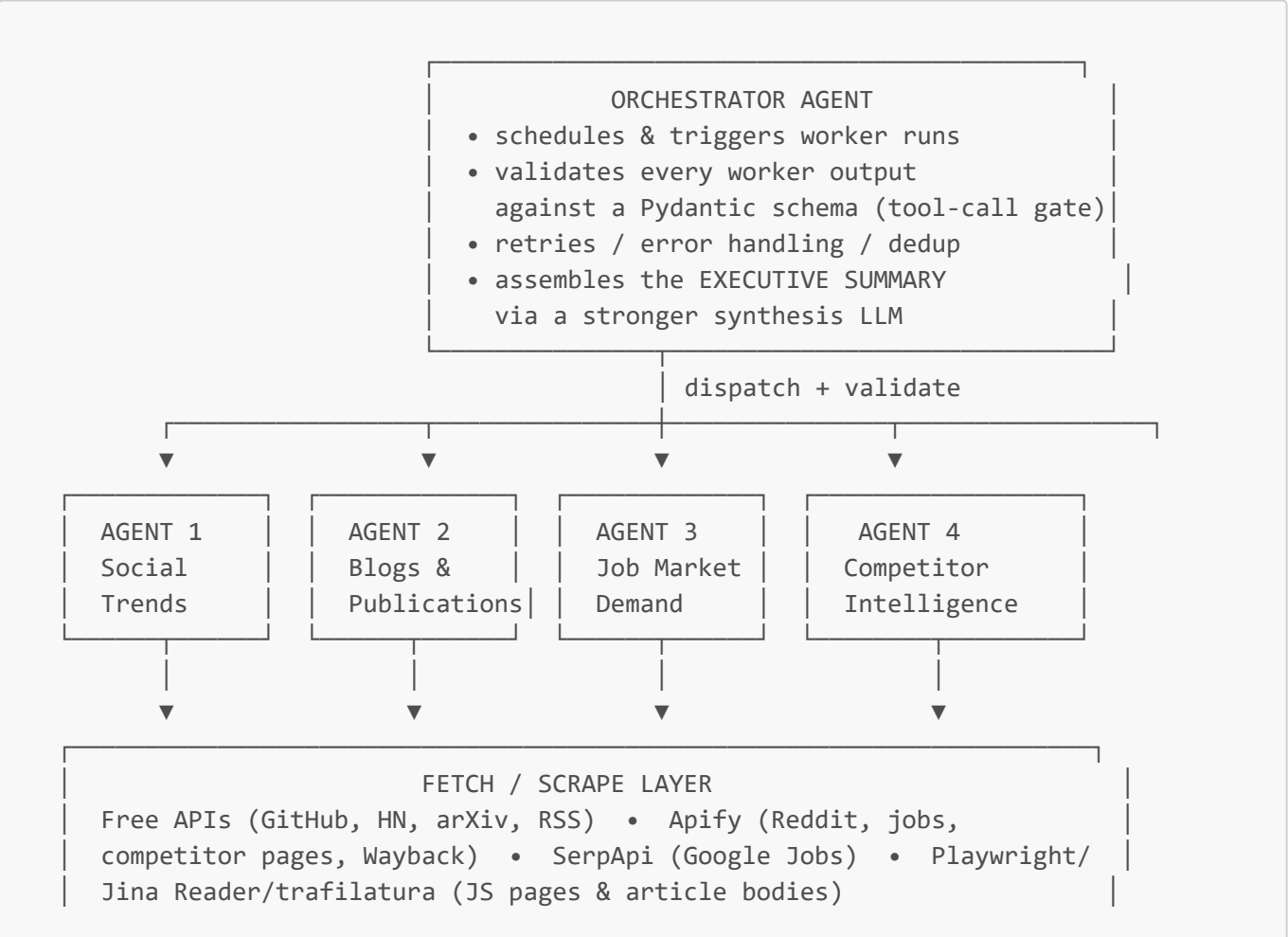
Build a **multi-agent system** = **4 worker agents + 1 orchestrator** that:

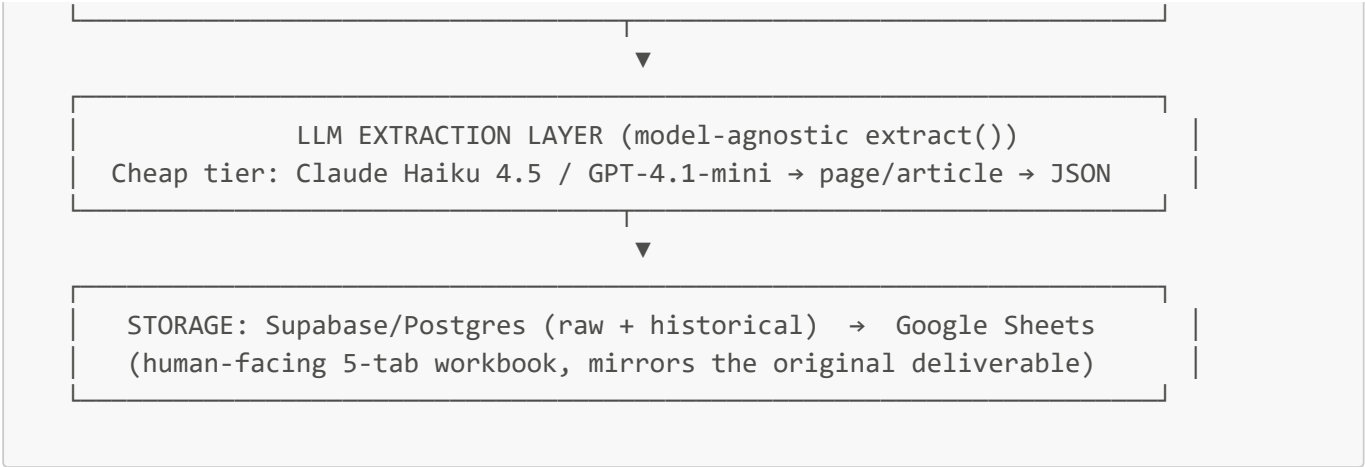
- Logs into / fetches / scrapes data from the same sources used in the original manual report.
- Extracts structured data per tab.
- Synthesises an Executive Summary (ranked strategic priorities) from the 4 tabs.
- Outputs a refreshed 5-tab workbook on a **twice-weekly** cadence.

Out of scope (v0.1): real-time alerting, a UI dashboard, anything requiring logged-in scraping of accounts we can't afford to lose (e.g., a primary LinkedIn account).

---

2. High-Level Architecture





---

3. Component Breakdown

3.0 Orchestrator Agent

- **Role:** the "brain" — does NOT scrape. It schedules, dispatches the 4 workers, validates their outputs, handles retries/dedup, and runs the Executive Summary synthesis.
- **Validation = the core primitive:** every worker returns data that must pass a **Pydantic schema** before it's accepted. Reject + retry on schema failure. This is the "validating the tool calls and directing them" requirement.
- **Executive Summary:** takes the 4 validated tab datasets, runs a **stronger LLM** (Sonnet 4.6 / Opus 4.8 / GPT-4.1) to produce the ranked strategic-priorities matrix.
- **Framework options: LangGraph** (state machine, checkpointing, retries) *or* **plain Python async + Pydantic** (minimal deps, deterministic). → *Recommendation: start with async + Pydantic; move to LangGraph only if state/branching grows.*

3.1 Agent 1 — Social Media Trends

Source	Fetch method	Cost
GitHub (stars, trending)	**Free** REST/GraphQL API + scrape `/trending`	$0
Hacker News	**Free** Firebase + Algolia API	$0
Reddit (r/developersIndia, r/MachineLearning, r/cscareerquestions)	**Apify Reddit scraper** ⭐ (avoids 2–4 wk official-API approval)	Apify credit
Product/version signals (Pinecone, Weaviate, MCP, Anthropic)	**Free** GitHub Releases + npm/PyPI JSON	$0
Twitter/X	⚠️ Pay-per-use only — **defer / optional**	—
LinkedIn, Quora	❌ Drop (no compliant free path)	—

Output: Source, Trending Topic, Sentiment (1–5), Engagement, Reference URL

3.2 Agent 2 — Blog & Publications

Source	Fetch method	Cost
arXiv	**Free** API / OAI-PMH (metadata is CC0)	$0
Towards Data Science, ML Mastery, YourStory, Entrepreneur	**Free** RSS / WordPress REST	$0
Analytics India Mag, AIM Research (JS SPAs)	sitemap crawl → **trafilatura** (Playwright if needed)	$0
Extraction: LLM maps article → {skill/tool, audience, urgency, curriculum-match}	Cheap LLM	small
Output: Publication, Title, Recommended Skill, Audience, Urgency, Curriculum Match, URL		

3.3 Agent 3 — Job Market Demand

Source	Fetch method	Cost
Naukri (native)	**Apify** Naukri actor (datacenter proxy OK)	Apify credit
Foundit	**Apify** Foundit actor	Apify credit
Glassdoor.co.in (salaries)	**Apify** Glassdoor actor (residential proxy)	Apify credit
Google Jobs (aggregates Indeed + LinkedIn)	**SerpApi** Google Jobs	SerpApi
HN "Who is hiring"	**Free** Algolia API	$0
Naukri JobSpeak Index	**Free** (macro hiring trend)	$0
Indeed / LinkedIn (direct)	❌ Skip — covered by Google Jobs aggregation	—
Note: Apify actors return **already-structured** rows (role, salary min/max, company) → minimal LLM needed here.		
Output: Portal, Role, Company Tier, Salary Min/Max (INR), YoY Demand, Reference URL		

3.4 Agent 4 — Competitor Intelligence

Source	Fetch method	Cost
Great Learning, Scaler, UpGrad, Udacity, Simplilearn (marketing pages)	**Jina Reader** (free) → **Apify** generic scraper fallback	mostly $0

Source	Fetch method	Cost
Hidden pricing (form-gated)	**Triangulate:** DOM/Jina → site XHR JSON → Shiksha/Careers360/CourseReport → Wayback	$0
Change detection	self-hosted **changedetection.io** + Wayback CDX (Apify)	low
Extraction: LLM → {program, duration, curriculum[], price_inr, emi, source, confidence}	Cheap LLM	small
Output: 14-column battlecard (program, price, duration, tools, credentials, where-we-win/lose, ad-intensity, URL)		

4. Tech Stack

Layer	Choice	Notes
Language	Python 3.11+	matches existing stack
Orchestration	Python async + Pydantic (v0.1) → LangGraph (if needed)	schema validation = the tool-call gate
Scraping providers	**Apify $29 Starter** (Reddit, Naukri, Foundit, Glassdoor, competitor, Wayback) + **SerpApi** (Google Jobs)	full rented-actor access
Free fetchers	`requests`/`httpx`, `feedparser`, `trafilatura`, GitHub/HN/arXiv APIs	for all non-hostile sources
JS rendering	**Jina Reader** (`r.jina.ai`, free) → Playwright fallback	competitor & SPA pages
LLM extraction (cheap tier)	**Claude Haiku 4.5** *or* **GPT-4.1-mini**, model-agnostic `extract()`	batched + prompt-cached
LLM synthesis (orchestrator)	**Sonnet 4.6 / Opus 4.8 / GPT-4.1**	Executive Summary only (~5 calls/run)
Storage	**Supabase/Postgres** (raw + historical) → **Google Sheets** (5-tab output)	Sheets mirrors original deliverable
Scheduling	**Apify Schedules** (cron) or own cron	twice weekly
Secrets	env vars / secrets manager; Playwright `storage_state` if any login	never commit

5. Pipeline / Data Flow (per run)

```
1. Orchestrator fires (cron, twice weekly)
2. Dispatches Agents 1–4 in parallel
3. Each agent:  fetch (Apify/SerpApi/free) → LLM extract → Pydantic-validate →
return rows
4. Orchestrator validates each agent's output; retries failures; dedups
5. Orchestrator runs Executive Summary synthesis (strong LLM) over the 4 datasets
6. Writes: Supabase (raw + versioned history) + Google Sheets (5 tabs)
7. Logs run metrics (rows, cost, failures)
```

---

6. Cost Model (monthly, ~9 runs)

Line item	Plan / rate	Est. / month
Apify	Starter $29 ($29 credit, $0.20/CU)	**$29** (hard floor)
SerpApi	Free 250 searches (Google Jobs, lean) — or $25 Starter if heavy	**$0–25**
LLM extraction (cheap tier, batched + cached)	Haiku $1/$5 or GPT-4.1-mini $0.40/$1.60 per 1M	**$2–9**
LLM synthesis (Exec Summary, ~5 calls/run)	Sonnet/Opus/GPT-4.1	**$2–11**
Supabase / Sheets / hosting	free tiers	**$0**
TOTAL		**≈ $33–60 / month**

Cost levers already baked in: Batch API (-50%), prompt caching (-up to 90% on repeated input), Apify result caps + datacenter proxies + scrape-only-tracked-roles, drop redundant portals (Indeed/LinkedIn covered by Google Jobs).

Apify $29 efficiency rules: set a $29 hard spend limit; vet each actor's *rental fee* before adding; cap `maxItems` per run; scrape only the ~20 tracked roles; lower actor memory; export outputs out of Apify immediately.

---

7. Compliance & Guardrails

- **Logged-out public data only.** Never bypass login walls (the legal line: *hiQ / Meta v. Bright Data* — logged-in scraping is actionable).
- **Prefer providers that warrant public-only sourcing** (Apify, SerpApi) so compliance burden shifts to them.
- **Don't auto-submit competitor lead-capture forms** — get gated pricing from aggregators/Wayback instead.
- **Don't store personal identities** from reviews/profiles (DPDP Act 2023).
- **Respect rate limits / robots.txt** on free sources; throttle politely.

- **No primary LinkedIn account in any agent.**

---

8. Phased Build Roadmap

Phase	Deliverable
0 — Prototype	Stand up ONE clean agent end-to-end (Agent 2 Blogs or HN/arXiv) on free tier; prove the extract→validate→Sheets path
1 — Fetch layer	Wire Apify ($29) actors (Reddit + jobs + competitor) + SerpApi; confirm structured output per portal
2 — All 4 agents	Build Agents 1–4 with Pydantic schemas; run each standalone
3 — Orchestrator	Validation gate, retries, dedup, parallel dispatch
4 — Exec Summary	Synthesis LLM produces ranked priorities matrix
5 — Output + schedule	Supabase + Google Sheets writers; Apify Schedule cron; cost/run logging
6 — Harden	Error alerting, change-detection for competitors, week-over-week diffing

---

9. Open Decisions (for our next review)

1. **Orchestration framework:** plain async+Pydantic vs LangGraph?
2. **Extraction model:** Haiku 4.5 vs GPT-4.1-mini (or both, A/B)?
3. **Synthesis model:** Sonnet 4.6 (cheaper) vs Opus 4.8 (smartest)?
4. **Twitter/X:** include (pay-per-use) or drop?
5. **Output target:** Google Sheets only, or Supabase + Sheets?
6. **SerpApi:** stay on free 250 or budget the $25 Starter?
7. **Cadence:** confirm twice-weekly (drives all cost math).
8. **Which exact Apify actors** (Reddit/Naukri/Foundit/Glassdoor) — confirm per-actor rental + per-result cost before subscribing.

Appendix A — Output Schemas (Pydantic)

These are the validation contracts the orchestrator enforces on every worker output. Columns mirror the original 5-tab workbook.

```python
from pydantic import BaseModel, Field, HttpUrl
from typing import Literal, Optional

# Tab 1 — Social Media Trends
class SocialTrend(BaseModel):
    source: str                       # GitHub / Reddit / HackerNews / ...
    topic: str
    sentiment: int = Field(ge=1, le=5)
```

```
    engagement: str                  # "122,000 stars" | "Viral" | ...
    reference_url: HttpUrl

# Tab 2 — Blog & Publications
class BlogPublication(BaseModel):
    publication: str
    title: str
    recommended_skill: str
    audience: str
    urgency: Literal["Must-have NOW", "High priority", "Future trend"]
    curriculum_match: Literal["Yes", "Partial", "No - GAP", "N/A"]
    reference_url: HttpUrl

# Tab 3 — Job Market Demand
class JobDemand(BaseModel):
    portal: str
    role: str
    company_tier: str
    salary_min_inr: int
    salary_max_inr: int
    yoy_demand: str                  # "+411%" | "High growth" | ...
    reference_url: HttpUrl

# Tab 4 — Competitor Intelligence
class Competitor(BaseModel):
    name: str
    program: str
    price_inr: Optional[int] = None
    price_source: str                # "Jina/DOM" | "Shiksha" | "Wayback" | ...
    confidence: Literal["high", "medium", "low"]
    duration_months: Optional[int] = None
    key_tools: list[str] = []
    credentials: Optional[str] = None
    where_we_win: list[str] = []
    where_they_win: list[str] = []
    ad_intensity: int = Field(ge=1, le=5)
    reference_url: HttpUrl

# Executive Summary — one ranked priority
class ExecutivePriority(BaseModel):
    rank: int
    direction: str
    sections: list[str]              # which tabs flagged it
    urgency: Literal["CRITICAL", "HIGH", "MEDIUM"]
    gap_vs_competitors: str
    actionable_insight: str
    effort_estimate: str
    revenue_impact: str
    supporting_data: str
```

Appendix B — Orchestrator: validate → retry → dedup (sketch)

```
import asyncio

async def run_agent(agent, schema, max_retries=2):
    for attempt in range(max_retries + 1):
        raw = await agent.fetch_and_extract()        # returns list[dict]
        rows, errors = [], []
        for item in raw:
            try:
                rows.append(schema(**item))          # ← the tool-call gate
            except ValidationError as e:
                errors.append((item, e))
        if not errors:
            return dedup(rows)
        # feed errors back to the agent's extractor for a corrective retry
        agent.add_correction_context(errors)
    return dedup(rows)                                # accept best-effort after
retries

async def orchestrate():
    social, blogs, jobs, comp = await asyncio.gather(
        run_agent(agent1, SocialTrend),
        run_agent(agent2, BlogPublication),
        run_agent(agent3, JobDemand),
        run_agent(agent4, Competitor),
    )
    summary = synthesize_exec_summary(social, blogs, jobs, comp)  # strong LLM
    write_supabase(social, blogs, jobs, comp, summary)
    write_google_sheets(social, blogs, jobs, comp, summary)       # 5 tabs
```

Appendix C — Model-agnostic `extract()`

Single function the agents call; swap the backend without touching agent logic.

```
def extract(page_text: str, schema: type[BaseModel], backend="haiku") -> dict:
    """Turn a rendered page / article into validated structured data."""
    system = build_extraction_prompt(schema)          # cached prefix
    if backend == "haiku":
        return claude_extract(system, page_text, model="claude-haiku-4-5")
    if backend == "gpt-mini":
        return openai_extract(system, page_text, model="gpt-4.1-mini")
    if backend == "ollama":
        return ollama_extract(system, page_text, model="llama3.2:3b")  # prototype
only
```

- Keep `system` (schema + instructions) **stable** so prompt caching kicks in (-up to 90% input cost).
- Run extraction calls through the **Batch API** (-50%) since the pipeline is scheduled, not interactive.

Appendix D — Apify Actor Shortlist (⚠️ confirm IDs + pricing on Apify Store before subscribing)

Need	Candidate actor (verify)	Billing model to check
Reddit (subreddits, posts, comments)	`trudax/reddit-scraper` (or `-lite`)	pay-per-result; no login needed
Naukri	`makework36/naukri-scraper`	pay-per-result (~$0.65–3.5/1K)
Foundit	`easyapi/foundit-jobs-scraper`	pay-per-result (~$2.99/1K)
Glassdoor.co.in	Apify Glassdoor salaries/jobs actor	per-result + residential proxy
Competitor pages	Apify generic web/Cheerio/Playwright scraper	per-CU; or use free Jina Reader first
Historical pricing	Apify Wayback Machine CDX actor	per-CU, cheap

Before subscribing, for each actor confirm: (1) monthly **rental fee** (if any), (2) per-result vs per-CU, (3) whether residential proxy is required. Rental fees are the main $29-budget risk.

Appendix E — Source → Connector Quick Reference (consolidated)

Source	Agent	Connector	Free?
GitHub	1	REST/GraphQL API + /trending scrape	✅
Hacker News	1, 3	Firebase + Algolia API	✅
Reddit	1	**Apify Reddit actor**	💲 Apify
Pinecone/Weaviate/MCP/Anthropic versions	1	GitHub Releases + npm/PyPI JSON	✅
arXiv	2	API / OAI-PMH (CC0)	✅
TDS / ML Mastery / YourStory / Entrepreneur	2	RSS / WP REST	✅
Analytics India Mag / AIM Research	2	sitemap → trafilatura	✅
Naukri	3	**Apify** + free JobSpeak Index	💲 Apify
Foundit	3	**Apify**	💲 Apify
Glassdoor.co.in	3	**Apify** (residential)	💲 Apify
Google Jobs	3	**SerpApi**	💲 /free 250
Great Learning / Scaler / UpGrad / Udacity / Simplilearn	4	Jina Reader → Apify fallback	mostly ✅

Source	Agent	Connector	Free?
Competitor pricing (gated)	4	Shiksha/Careers360/CourseReport + Wayback	✅
Twitter/X	1	pay-per-use API (optional/deferred)	❌
LinkedIn / Quora / Indeed (direct)	—	dropped (compliance/redundant)	—

End of draft v0.1 — numbers and choices are provisional and to be confirmed in review.